

Time and Space Complexity Analysis

Unrestricted Edit Distance Algorithm

1. Function: NeedlemanWunsch

Time Complexity: $O(n * m)$

Space Complexity: $O(n * m)$

```
table = [[0] * (len(seq2) + 1) for i in range(len(seq1) + 1)] # O(n * m) time, O(n * m) space
```

```
back = [[None] * (len(seq2) + 1) for i in range(len(seq1) + 1)] # O(n * m) time, O(n * m) space
```

```
for j in range(len(seq2)+1): # m iterations
```

<code>table[0][j] = j * INDEL</code>	<code># O(1)</code>
--------------------------------------	---------------------

```
for i in range(len(seq1)+1): # n iterations
```

$$table[i][0] = i * INDEL \quad \# O(1)$$

```
for i in range(1, len(seq1)+1):           # n iterations
```

```
for j in range(1, len(seq2)+1):           # m iterations
```

```
diag = table[i-1][j-1] + (MATCH if seq1[i-1] == seq2[j-1] else SUB) # O(1)
```

$$top = table[i-1][j] + INDEL \quad \# O(1)$$
$$left = table[i][j-1] + INDEL \quad \# O(1)$$

```
table[i][j] = min(left, top, diag)      # O(1)
```

```
if table[i][j] == left: # O(1)
```

```
back[i][j] = "I" # O(1)
```

```
elif table[i][j] == top: # O(1)
```

```
back[i][j] = "t" # O(1)
```

else:

```
back[i][j] = "d" # O(1)
```

total: $n * m$ iterations * $O(1) = O(n * m)$

```
i = len(seq1) # O(1)
```

```
j = len(seq2) # O(1)
```

```
alignment1 = "" # O(1)
```

```
alignment2 = "" # O(1)
```

```

while i > 0 or j > 0:                                # at most n + m iterations
    direction = back[i][j]                            # O(1)
    if direction == "l":
        alignment1 = "-" + alignment1                # O(1)
        alignment2 = seq2[j-1] + alignment2          # O(1)
        j -= 1                                        # O(1)
    elif direction == "t":
        alignment1 = seq1[i-1] + alignment1          # O(1)
        alignment2 = "-" + alignment2                # O(1)
        i -= 1                                        # O(1)
    else:
        alignment1 = seq1[i-1] + alignment1          # O(1)
        alignment2 = seq2[j-1] + alignment2          # O(1)
        i -= 1                                        # O(1)
        j -= 1                                        # O(1)
# total: O(n + m)

```

Time complexity:

Initialization: $O(n * m)$

DP fill: $n * m * O(1) = O(n * m)$

Backtracking: $O(n + m)$

Total:

$$T(n, m) = O(n * m) + O(n * m) + O(n + m) = O(n * m)$$

Space complexity:

table : $O(n * m)$

back pointers table : $O(n * m)$

alignment strings: $O(n + m)$

Total:

$$S(n, m) = O(n * m)$$

2. Function: NeedlemanWunschBanded

Time Complexity: $O(k * n)$

Space Complexity: $O(k * n)$

```

if abs(len(seq1) - len(seq2)) > MAXINDELS:      # O(1)

    return math.inf, "", ""                    # O(1)

table = [[float('inf')] * (2*MAXINDELS+1) for i in range(len(seq1) + 1)] # O(n * k) time, O(n * k) space

back = [[None] * (2*MAXINDELS+1) for i in range(len(seq1) + 1)]        # O(n * k) time, O(n * k) space

table[0][MAXINDELS] = 0                                # O(1)

for i in range(len(seq1) + 1):                          # n iterations

    for banded_j in range(2 * MAXINDELS + 1):          # k iterations

        j = i + (banded_j - MAXINDELS)                # O(1)

        if j < 0 or j > len(seq2):                    # O(1)

            continue                                    # O(1)

        if i == 0 and j == 0:                          # O(1)

            continue                                    # O(1)

        diag = float('inf')                            # O(1)

        top = float('inf')                             # O(1)

        left = float('inf')                             # O(1)

        if i > 0 and j > 0:                             # O(1)

            cost = MATCH if seq1[i-1] == seq2[j-1] else SUB # O(1)

            diag = table[i-1][banded_j] + cost          # O(1)

            if i > 0 and banded_j + 1 < 2 * MAXINDELS + 1: # O(1)

                top = table[i-1][banded_j+1] + INDEL    # O(1)

            if j > 0 and banded_j - 1 >= 0:              # O(1)

                left = table[i][banded_j-1] + INDEL    # O(1)

            table[i][banded_j] = min(left, top, diag)    # O(1)

            if table[i][banded_j] == left:              # O(1)

                back[i][banded_j] = "I"                # O(1)

            elif table[i][banded_j] == top:              # O(1)

```

```

        back[i][banded_j] = "t"                                # O(1)

    else:

        back[i][banded_j] = "d"                                # O(1)

# total: n * k iterations * O(1) = O(n * k)

i = len(seq1)                                                  # O(1)
j = len(seq2)                                                  # O(1)

banded_j = MAXINDELS + (j - i)                                # O(1)

alignment1 = ""                                                # O(1)

alignment2 = ""                                                # O(1)

while i > 0 or j > 0:                                          # at most n + m iterations

    direction = back[i][banded_j]                              # O(1)

    if direction == "d":

        alignment1 = seq1[i-1] + alignment1                  # O(1)

        alignment2 = seq2[j-1] + alignment2                  # O(1)

        i -= 1                                                # O(1)

        j -= 1                                                # O(1)

        banded_j = MAXINDELS + (j - i)                        # O(1)

    elif direction == "t":

        alignment1 = seq1[i-1] + alignment1                  # O(1)

        alignment2 = "-" + alignment2                        # O(1)

        i -= 1                                                # O(1)

        banded_j += 1                                         # O(1)

    elif direction == "l":

        alignment1 = "-" + alignment1                        # O(1)

        alignment2 = seq2[j-1] + alignment2                  # O(1)

        j -= 1                                                # O(1)

        banded_j -= 1                                         # O(1)

```

total: $O(n + m)$

Time complexity:

Initialization:

$O(n * k)$

DP band computation:

$n * k * O(1) = O(n * k)$

Backtracking:

$O(n + m)$

Total:

$$T(n) = O(n * k) + O(n * k) + O(n + m) = O(n * k)$$

Space complexity:

table : $O(n * k)$

back pointers: $O(n * k)$

alignment strings : $O(n + m)$

Total:

$$S(n) = O(n * k)$$

Alignment Extraction and Backtrace Explanation

The alignment extraction algorithm reconstructs an optimal alignment by performing a backtrace through the back pointer table generated during dynamic programming. After the scoring table is fully computed, the algorithm begins at the bottom-right cell, which represents the optimal alignment cost for the full sequences. At each step, the algorithm consults the corresponding entry in the back pointer table to determine the direction of the optimal move: left ("l"), top ("t"), or diagonal ("d"). A diagonal move indicates either a match or substitution, so both characters from the sequences are added to the alignment. A left move represents an insertion (gap in the first sequence), so a "-" is added to the first alignment and the corresponding character from the second sequence is added. A top move represents a deletion (gap in the second sequence), so a character from the first sequence is added and a "-" is added to the second alignment. The indices are updated accordingly after each step, and the process continues until both indices reach zero. Since the alignment is constructed in reverse order, characters are appended to the beginning at each step to produce the final aligned sequences. The same backtrace procedure is used for both the unrestricted and banded algorithms, with the only difference being that the banded version maps indices within a restricted diagonal band of width k .

Results

Unrestricted results:

	sequence1	sequence2	sequence3	sequence4	sequence5	sequence6	sequence7	sequence8	sequence9	sequence10
sequence1	-30	-1	4956	4956	4956	4956	4956	4956	4956	4956
sequence2		-33	4948	4948	4948	4948	4948	4948	4948	4948
sequence3			-3000	-2996	-2956	-2944	-1431	-1448	-1399	-1448
sequence4				-3000	-2960	-2948	-1431	-1448	-1399	-1448
sequence5					-3000	-2988	-1423	-1452	-1391	-1448
sequence6						-3000	-1426	-1452	-1394	-1448
sequence7							-3000	-2771	-2814	-2767
sequence8								-3000	-2731	-2996
sequence9									-3000	-2727
sequence10										-3000

Label I:

Sequence I:

Sequence J:

Label J:

☐ Banded Align Length:

Done. Time taken: 11.554 seconds.

Banded results:

	sequence1	sequence2	sequence3	sequence4	sequence5	sequence6	sequence7	sequence8	sequence9	sequence10
sequence1	-30	-1	inf	inf	inf	inf	inf	inf	inf	inf
sequence2		-33	inf	inf	inf	inf	inf	inf	inf	inf
sequence3			-9000	-8984	-8888	-8848	-2735	-2743	-1429	-2735
sequence4				-9000	-8888	-8848	-2739	-2748	-1426	-2740
sequence5					-9000	-8960	-2711	-2739	-1426	-2727
sequence6						-9000	-2708	-2728	-1415	-2716
sequence7							-9000	-8103	-1256	-8099
sequence8								-9000	-1310	-8980
sequence9									-9000	-1315
sequence10										-9000

Label I:

Sequence I:

Sequence J:

Label J:

☒ Banded Align Length:

Unrestricted algorithm:

gattgcgagcgatttgcgtgcgtgcatcccgcttc-actg--at-ctcttgtagatcttttcataatctaaactttataaaaaatccactccctgta-
gataa-gagtgattggcggtccgtacgtaccctttctactctcaaactcttgtagtttaaatac-taatctaaactttataaa--cggc-acttcctgtgt

Banded algorithm:

gattgagcgatttgcgtgcgtgcatcccgcttc-actg--at-ctcttgtagatctttcataatctaaactttataaaaaacatccactccctgta-
-ataa-gagtgattggcggtccgtacgtaccctttctactctcaaactcttgtagtttaaadc-taatctaaactttataaa--cggc-acttcctgtgt

GitHub link:

[AdvancedAlgorithmsCS4412/proj4 at master · akkabakovb/AdvancedAlgorithmsCS4412](#)